

Projeto Único: Sistema de Gestão Hospitalar Dra. Yuska Maritan Brito

Contexto: O Hospital Universitário Dra. Yuska Maritan Brito precisa de um sistema para gerenciar atendimentos, profissionais, pacientes, procedimentos, internações e escalas de plantão.

O sistema deve cadastrar Pessoas. Toda pessoa possui nome, CPF, data de nascimento, is_flamengo e telefone. Uma pessoa pode ser Paciente (com atributos: número do convênio, alergias, grupo sanguíneo) ou Profissional (com atributos: CRM, data de admissão, especialidade). Um Profissional pode ser Preceptor (médico responsável) ou Residente (médico em formação). Um residente possui um atributo "ano_residencia" (R1, R2, R3). Um preceptor possui um atributo "titulacao" (mestre, doutor, etc.). Um profissional pode atuar como preceptor em um determinado período e como residente em outro (histórico), mas em um dado momento ele ocupa apenas um papel no sistema.

Um Atendimento ocorre em uma data e horário específicos, com duração registrada em minutos. Em cada atendimento, há exatamente um paciente, um residente (que realiza o atendimento sob supervisão) e um preceptor (que supervisiona aquele atendimento específico). Durante um atendimento, podem ser realizados um ou mais procedimentos (ex: sutura, coleta de sangue, aplicação de medicação). Cada procedimento possui um código, nome e tempo médio de execução. Para cada procedimento realizado em um atendimento, registra-se a quantidade executada, o tempo real gasto e uma observação sobre intercorrências.

O hospital possui Unidades (Enfermaria, UTI, Pronto-Socorro, Ambulatório). Os residentes e preceptores se organizam em Escalas de Plantão. Em uma escala, define-se: uma unidade, um dia da semana (segunda a domingo), um turno (manhã, tarde, noite), um residente e um preceptor responsável pela supervisão naquele plantão. Uma combinação de unidade, dia, turno, residente e preceptor é única (não pode haver o mesmo residente no mesmo local/dia/turno com dois preceptores distintos). O mesmo preceptor pode supervisionar vários residentes no mesmo plantão (desde que em unidades ou turnos diferentes), mas para cada residente registra-se um único preceptor supervisor por plantão.

O sistema será desenvolvido em duas etapas, com complexidade crescente.

Objetivos de Aprendizagem:

- Modelagem conceitual, lógica e física (DER → MR → SQL)
- Normalização até 3FN/BCNF
- SQL avançado (junções, subconsultas, agregações, views)
- Triggers e stored procedures
- Uso de ORM (SQLAlchemy, Prisma, Hibernate, Entity Framework, ou similar)
- Controle de transações e integridade referencial

Tecnologias sugeridas: PostgreSQL ou MySQL (backend), Python com SQLAlchemy (ou Node.js com Prisma, Java com Hibernate, C# com EF Core). Frontend livre (CLI, web, ou desktop).

Etapa 1 (Entrega: 4-5 semanas) – Fundamentos

Objetivo

Implementar o modelo relacional básico, operações CRUD e consultas essenciais, sem ainda usar ORM (SQL puro).

```
PESSOA (id_pessoa PK, nome, CPF, data_nascimento, is_flamengo, telefone)
├── PACIENTE (id_pessoa PK → PESSOA, num_convenio, alergias, grupo_sanguineo)
├── PROFISSIONAL (id_pessoa PK → PESSOA, CRM, data_admissao, especialidade)
│   ├── PRECEPTOR (id_profissional PK → PROFISSIONAL, titulacao)
│   └── RESIDENTE (id_profissional PK → PROFISSIONAL, ano_residencia)

UNIDADE (id_unidade PK, nome, tipo, capacidade_leitos)

ATENDIMENTO (id_atendimento PK, data_hora, duracao_minutos,
             id_paciente FK → PACIENTE,
             id_residente FK → RESIDENTE,
             id_preceptor FK → PRECEPTOR)

PROCEDIMENTO (id_procedimento PK, codigo, nome, tempo_medio_minutos)

PROCEDIMENTO_REALIZADO (id_atendimento FK, id_procedimento FK,
                       quantidade, tempo_real_minutos, observacao,
                       PK composta)

ESCALA (id_escalas PK, id_unidade FK, dia_semana, turno,
        id_residente FK, id_preceptor FK,
        UNIQUE(id_unidade, dia_semana, turno, id_residente))
```

Requisitos da Etapa 1

1. Modelagem (2 pontos)

- DER completo (entregar em PDF com justificativas de cardinalidades e especialização)
- Modelo relacional (todas as tabelas, chaves primárias e estrangeiras)
- Evidência de normalização até 3FN (justificar)

2. Implementação do BD (3 pontos)

- Script SQL para criação de todas as tabelas (CREATE TABLE com constraints: PK, FK, CHECK, NOT NULL, UNIQUE)
- Inserção de dados de teste (mínimo: 5 pacientes, 5 residentes, 5 preceptores, 3 unidades, 10 atendimentos, 10 procedimentos realizados)

3. CRUD e consultas básicas (3 pontos – via SQL puro, sem ORM)

- Inserir um novo atendimento (verificando se paciente, residente, preceptor existem)

- Listar todos os atendimentos de um paciente específico (ordenados por data)
- Listar os procedimentos realizados em um atendimento (com nome do procedimento, quantidade e tempo real)
- Atualizar os dados de um paciente (endereço ou convênio)
- Remover um procedimento realizado (apenas se ainda não houver faturamento associado – usar uma flag)
- Calcular o tempo médio de duração dos atendimentos por residente

4. Consultas analíticas (2 pontos – SQL puro)

- Ranking dos residentes por número de atendimentos realizados (mostrar nome e total)
- Listar os preceptores que supervisionaram mais de 5 atendimentos em um determinado mês
- Para cada unidade, mostrar a quantidade de plantões escalados por residente no mês corrente
- Listar pacientes que nunca realizaram nenhum procedimento de nível de risco 'ALTO'

5. Documentação e apresentação (1 ponto extra)

- README com instruções de instalação e execução dos scripts
- Apresentação de 10 minutos demonstrando as funcionalidades

Etapa 2 (Entrega: +4-5 semanas) – Funcionalidades Avançadas

Objetivo

Adicionar regras de negócio via stored procedures, triggers, views, migrar a aplicação para uma ORM e implementar transações complexas.

Novos Requisitos

1. Stored Procedures (1,5 ponto)

- **sp_registrar_atendimento_completo**: recebe dados do atendimento + lista de procedimentos realizados (como JSON ou tabela temporária) e insere tudo dentro de uma **transação** (se qualquer procedimento falhar, tudo é revertido).
- **sp_calcular_tempo_medio_espera**: calcula, para cada unidade, o tempo médio entre a chegada do paciente (data_hora do atendimento) e o início do primeiro procedimento.
- **sp_reajustar_escala**: recebe um id_residente, muda todas as suas escalas de um dia/turno para outro, desde que não gere conflito (mesmo unidade+dia+turno+residente).

2. Triggers (1,5 ponto)

- **trg_check_sobreposicao_escala**: BEFORE INSERT/UPDATE na tabela ESCALA. Impede que um mesmo residente seja escalado no mesmo dia/turno em duas unidades diferentes.
- **trg_audita_atendimento**: AFTER INSERT/UPDATE/DELETE em ATENDIMENTO. Registra em uma tabela **AUDITORIA_ATENDIMENTO** (id_auditoria, id_atendimento, operacao, usuario, data_hora, dados_antigos (JSON), dados_novos (JSON)).
- **trg_atualiza_media_procedimentos**: AFTER INSERT em PROCEDIMENTO_REALIZADO. Atualiza uma coluna **media_tempo_procedimento** na tabela PROCEDIMENTO (média do tempo_real_minutos daquele procedimento em todos os atendimentos).

3. Views (1,0 ponto)

- **vw_pacientes_internados**: pacientes que estão atualmente internados (data_hora_saida IS NULL na internação mais recente).
- **vw_residentes_sem_supervisor**: residentes que estão escalados em algum plantão, mas cujo preceptor não tem titulação de doutor (ou não possui supervisão ativa).
- **vw_estatisticas_atendimentos_mensal**: agregação por mês e por unidade: total de atendimentos, média de duração, procedimentos mais comuns.

4. ORM (2,0 pontos)

- Reimplementar **todas** as operações da Etapa 1 usando uma ORM à escolha:
 - **Python**: SQLAlchemy (recomendado) ou Django ORM
 - **Node.js**: Prisma ou TypeORM
 - **Java**: Hibernate
 - **C#**: Entity Framework Core
- Demonstrar:
 - Mapeamento objeto-relacional (classes/entidades)
 - Uso de sessões/transações via ORM
 - Consultas usando a DSL/filter da ORM (não SQL cru)
 - Relacionamentos (lazy loading vs eager loading)

5. Consultas avançadas com ORM (1,0 ponto)

- Usando a ORM, implemente:
 - Listar todos os preceptores que supervisionaram residentes que atenderam pacientes que são flamenguistas (**is_flamengo = TRUE**).
 - Para cada paciente, exibir seu último atendimento (data_hora, residente, preceptor, lista de procedimentos).

- Calcular o percentual de procedimentos de alto risco realizados por cada residente.

6. Tratamento de concorrência e transações (1,0 ponto)

- Implementar um cenário simulado de **duas transações concorrentes** tentando escalar o mesmo residente para o mesmo dia/turno/unidade.
- Usar mecanismos da ORM/BD para evitar inconsistência (lock otimista ou pessimista).
- Demonstrar com código e logs.

7. Entrega Final (1 ponto extra)

- Repositório GitHub com commits separados por Etapa 1 e Etapa 2.
- Vídeo de até 8 minutos demonstrando as novas funcionalidades da Etapa 2.
- Relatório breve (2 páginas) explicando as decisões de implementação, especialmente sobre triggers vs procedures e escolha da ORM.